

Federated Steering on Newsroom: Experimental Settings and Results

Anonymous ACL submission

1 Experimental Setup

1.1 Task, attribute and clients

Task. We study single-document news summarization on Newsroom (Grusky et al., 2018), and steer the *extractiveness* of the generated summary. Extractiveness is measured by fragment density (Grusky et al., 2018). Let $\mathcal{F}(A, S)$ be the set of maximal shared token sequences (extractive fragments) between article A and summary S . Then

$$\text{density}(A, S) = \frac{1}{|S|} \sum_{f \in \mathcal{F}(A, S)} |f|^2. \quad (1)$$

Density is low when a summary rewrites its source, and high when it copies long spans. We reimplemented the scorer with a regex tokenizer. Its Spearman correlation with the density values distributed with Newsroom is 0.99. Articles are truncated to 400 words, and density is recomputed on the truncated article. Pairs whose summary depends on the removed text are dropped. The same scorer labels the training data and evaluates generated summaries.

Clients. Each publication is a client. We take the 12 publications with at least 5,000 usable pairs whose median densities are evenly spaced, from telegraph.co.uk (median 1.3) to nypost.com (32.0). Table 1 lists them. The publications are split into three held-out rotations of four clients, stratified by density. All results here use rotation 0: eight *participants* train the shared control mechanism, and four *held-out* clients join afterwards. The publications simulate organizations; this is not a deployment involving the publishers.

Splits. Each client has a training pool of up to 5,000 pairs, a development split of 100 articles, and a test split of 200 articles taken from Newsroom’s official test split (180 for theguardian.com). Participants train on 4,000 pairs each unless stated

Publication	Role	Median dens.	Pool	Support
forbes.com	part.	1.80	5,000	[0.03, 0.99]
theguardian.com	part.	1.73	4,705	[0.04, 0.55]
people.com	part.	2.07	5,000	[0.03, 0.70]
wsj.com	part.	5.14	5,000	[0.04, 0.89]
aol.com	part.	9.09	5,000	[0.06, 0.82]
cbc.ca	part.	9.89	5,000	[0.11, 0.95]
reuters.com	part.	20.00	4,023	[0.10, 0.97]
nypost.com	part.	32.03	5,000	[0.51, 0.95]
telegraph.co.uk	held-out	1.26	5,000	[0.02, 0.99]
bbc.com	held-out	2.20	4,228	[0.03, 0.79]
mashable.com	held-out	4.88	5,000	[0.08, 0.79]
latimes.com	held-out	18.65	5,000	[0.11, 0.98]

Table 1: Clients of rotation 0. *Median dens.*: median fragment density of the training pool. *Pool*: training pairs available. *Support*: 5th–95th percentile of the client’s own data on the shared control scale α (Section 1.2). theguardian.com lacks extractive summaries; nypost.com lacks abstractive ones.

otherwise. Training, development and test data come from years before each client’s latest year, which is held back. That later data is not used in the experiments reported here.

Data characteristics. Two properties of the raw data matter for interpretation. First, 92% of ny-post.com’s raw summaries are its first ~ 200 characters followed by an ellipsis, a fixed format taken from page metadata. We keep them, and treat ny-post.com as a client with a hard format convention. Second, density is partly coupled to length (Spearman 0.64 on the training data), so we report output length as an off-target measure.

1.2 Shared control scale

A requested level $\alpha \in [0, 1]$ is a percentile on a scale shared by all clients. Let $a(y)$ be the density of summary y and F_i the empirical distribution of a on participant i ’s training data. The reference distribution is the equal-weight mixture $F = \frac{1}{M} \sum_{i=1}^M F_i$ over the $M = 8$ participants. The server can build F by averaging normalized per-client histograms, so no examples leave a client.

Every training pair is labelled $\alpha(y) = F(a(y))$, and held-out clients reuse the same F . Under this scale, $\alpha = 0.5$ requests the same behaviour on every client. A client’s data covers only part of $[0, 1]$. Its *support* is the interval between the 5th and 95th percentiles of its own labels. Requests outside the support are *out-of-support*. Values there are scarce in the client’s data, although not necessarily absent.

1.3 Model and federated training

Parameterization. Every attention and MLP projection with frozen weight W_0 of client i computes

$$y = W_0 x + B_i^P A_i^P x + g(\alpha) B^D A^D x. \quad (2)$$

$B_i^P A_i^P$ is a private rank-16 LoRA adapter that never leaves the client. $B^D A^D$ is the shared rank-16 steering direction D . A^D is a random matrix generated from a common seed and kept frozen; only B^D is trained, starting at zero. Since A^D is identical across clients, averaging B^D averages the direction products exactly. The calibration g maps the requested level to an intervention strength:

$$g(\alpha) = s h(\alpha), \quad (3)$$

$$h(\alpha) = (1 - w) \alpha + w [1 - (1 - \alpha^p)^q]. \quad (4)$$

Here $s = \exp(u) \in [\frac{1}{4}, 4]$ is a gain, and h is a monotone map with $h(0) = 0$ and $h(1) = 1$, initialized at the identity. The calibration is *shared*: one (u, w, p, q) for all clients, aggregated with B^D . There is no offset, so $g(0) = 0$.

Objective. Each client minimizes the negative log-likelihood of its reference summaries with steering active at the summary’s own level,

$$\mathcal{L}_i = \mathbb{E}_{(x,y) \sim S_i} [-\log p_{W_i(\alpha(y))}(y | x)], \quad (5)$$

over B_i^P, A_i^P , its copy of B^D and the calibration. Task adaptation and control are therefore learned in a single loss, and the direction is trained in the presence of the private adapters it must act on.

Federated protocol. We use FedAvg with full participation of the eight participants. In each round, every client takes 20 local AdamW steps with batch size 8. The server then averages B^D and the calibration parameters uniformly across clients, so large clients do not dominate. Private adapters and their optimizer states persist across rounds; the optimizer state of shared parameters is reset each round. Training runs for 100 rounds. Learning

rates are 2×10^{-4} for the adapter and direction and 10^{-2} for the gain and warp, with 20 warm-up steps. The gain is trained from round 2 and the warp from round 5, with a penalty $0.01 \cdot \overline{(h(a) - a)^2}$ toward the identity. Prompts are truncated to 1,024 tokens and targets to 128 tokens.

Backbones. The main backbone is Llama-3.2-1B-Instruct. We replicate the participant comparison on Qwen3-8B, with thinking disabled. Both use identical data and hyperparameters.

1.4 Aligned calibration

A calibration shared by parameter averaging fits one curve for all clients, yet a client contributes useful evidence only where it has data. A per-client calibration uses that evidence but must extrapolate outside the client’s support, where it fails for skewed clients (Section 2.4). *Aligned calibration* keeps one calibration curve per client and layer, but every forward pass, in training and at inference, uses a single pooled curve. In this pooled curve, each region of the α axis is determined by the clients that have data there.

Per-layer shapes. The gain is fixed at $s = 1$ and there is no offset. Client i has its own monotone shape $h_{i,l}$ of the form above for every adapted matrix l , with $h_{i,l}(0) = 0$ and $h_{i,l}(1) = 1$. Shape parameters are never averaged. Instead, the server pools their *values* on a common grid $a_k = \frac{k-1}{K-1}$, $k = 1, \dots, K$.

Evidence weights. Client i ’s evidence at grid point a_k is a triangular-kernel count of its training labels,

$$c_{ik} = \sum_{(x,y) \in S_i} \max\left(0, 1 - \frac{|\alpha(y) - a_k|}{b}\right), \quad (6)$$

computed once from the client’s training subset. It enters the pool through the saturating weight

$$\omega_{ik} = \frac{c_{ik}}{\tau + c_{ik}} \in [0, 1). \quad (7)$$

The weight approaches 1 wherever the client has much nearby data and is 0 where it has none. A dense client therefore cannot dominate a pooled value, in keeping with the equal-weight reference F .

Pooled curve. For layer l , the pooled value at a_k is the evidence-weighted mean of the clients’

curves,

$$m_{l,k} = \frac{1}{\Omega_k} \sum_j \omega_{jk} h_{j,l}(a_k), \quad \Omega_k = \sum_j \omega_{jk}. \quad (8)$$

Ω_k is the *total evidence weight* at a_k : the sum of all participants' saturating weights there. It depends only on the counts c_{jk} , which are fixed before training, so it is the same for every client and every layer, and it is a constant in all gradients below. We call a_k *covered* if $\Omega_k > 0$, i.e. if at least one client has data within distance b of it. The covered grid points, in increasing order, form the set $\mathcal{C} = \{k : \Omega_k > 0\} = \{k_1 < k_2 < \dots < k_n\}$. The pooled calibration is the monotone projection $\bar{g}_l = \Pi(m_l)$ of these values:

- on the covered points, Π is the weighted isotonic regression

$$(\bar{g}_l(a_k))_{k \in \mathcal{C}} = \arg \min_z \sum_{k \in \mathcal{C}} \Omega_k (z_k - m_{l,k})^2 \quad (9)$$

subject to $z_{k_1} \leq z_{k_2} \leq \dots \leq z_{k_n}$, so a pooled value backed by more evidence moves less when monotonicity has to be restored;

- at a grid point no client covers ($\Omega_k = 0$), the pooled mean is undefined, so Π *defines* the value by linear interpolation between the nearest covered points $a_{k-} < a_k < a_{k+}$ (value 0 or 1 if there is no covered point on one side);
- endpoints set exactly to $\bar{g}_l(0) = 0$ and $\bar{g}_l(1) = 1$.

Between grid points, $\bar{g}_l(\alpha)$ is the piecewise-linear interpolation of the table. The coefficient on the direction in layer l of Equation 2 is then $\bar{g}_l(\alpha)$ for every client. This pools function values, not parameters. Where only one client has data, $\bar{g}_l$ follows that client's curve. Where several overlap, $\bar{g}_l$ is their weighted average.

Training through the pooled curve. To use the same map in training and at inference, client i trains *through* $\bar{g}$. In each local step it rebuilds the pooled curve from its own current shape values, which carry gradients, and the other clients' values from the end of the previous round, $\tilde{h}_{j,l}$, which are held fixed:

$$m_{l,k}^{(i)} = \frac{\omega_{ik} h_{i,l}(a_k) + \sum_{j \neq i} \omega_{jk} \tilde{h}_{j,l}(a_k)}{\sum_j \omega_{jk}}, \quad (10)$$

$$\bar{g}_l^{(i)} = \Pi(m_l^{(i)}). \quad (11)$$

The server sends client i only the leave-one-out sums $\sum_{j \neq i} \omega_{jk} \tilde{h}_{j,l}(a_k)$ and $\sum_{j \neq i} \omega_{jk}$. After a round, clients upload their grid values $h_{i,l}(a_k)$. The server stores them as the next round's $\tilde{h}$ and builds the inference table by applying the same operator to all clients' final values. The map a client trains through thus equals the inference map, except that the other clients' contributions are one round old.

Gradient through the pooled curve. In client i 's step, the other clients' values $\tilde{h}_{j,l}$ are constants. Only client i 's shape parameters $\psi_{i,l} = (w_{i,l}, p_{i,l}, q_{i,l})$ receive gradient through $\bar{g}$. The adapter and the direction receive their usual gradients, with $\bar{g}_l^{(i)}(\alpha)$ acting as a multiplier. The pooled curve is rebuilt from the current $\psi_{i,l}$ before every forward pass. The gradient of the loss with respect to $\psi_{i,l}$ is the composition of four steps, listed from the loss backwards.

(1) *Interpolation.* A training pair with level $\alpha \in [a_k, a_{k+1}]$ uses $\bar{g}_l^{(i)}(\alpha) = (1 - \rho) \bar{g}_l^{(i)}(a_k) + \rho \bar{g}_l^{(i)}(a_{k+1})$, where $\rho = (\alpha - a_k)(K - 1)$. The coefficient's gradient, $\partial \ell / \partial \bar{g}_l^{(i)}(\alpha) = \sum_t \langle \partial \ell / \partial y_t, B^D A^D x_t \rangle$ summed over token positions t , is split between the two neighbouring grid values with weights $1 - \rho$ and ρ . Summed over the batch, this gives $G_{l,k} = \partial \mathcal{L}_i / \partial \bar{g}_l^{(i)}(a_k)$. At a grid point that no client covers, Π sets the value to $(1 - t) \bar{g}_l^{(i)}(a_{k-}) + t \bar{g}_l^{(i)}(a_{k+})$, with $t = (a_k - a_{k-}) / (a_{k+} - a_{k-})$. This is part of the definition of Π , not an approximation, and the inference table uses the same rule. The value is a fixed linear function of its two neighbours, so it passes its gradient to them with weights $1 - t$ and t . Any value between the two neighbours keeps $\bar{g}$ monotone; linear interpolation additionally makes $\bar{g}$ a single straight line between a_{k-} and a_{k+} , as if the uncovered points were absent from the grid. On Newsroom no grid point is uncovered: every a_k has a total smoothed count above 3,000. The endpoints are the constants 0 and 1 and pass none.

(2) *Projection.* The solution of Equation 9 has a block structure, which defines $B(k)$. A *block* is a set of covered points that are consecutive in $\mathcal{C}$, $B = \{k_r, k_{r+1}, \dots, k_s\}$; it may span uncovered grid points, which carry no weight. A block's weight and weighted mean are

$$\Omega_B = \sum_{k \in B} \Omega_k, \quad \mu(B) = \frac{1}{\Omega_B} \sum_{k \in B} \Omega_k m_{l,k}^{(i)}. \quad (12)$$

The pool-adjacent-violators algorithm partitions $\mathcal{C}$

into blocks $B_1, \dots, B_r$, ordered from left to right. It starts with every covered point as its own block. Scanning from left to right, whenever a block's mean is strictly greater than the mean of the block to its right, it merges the two, and it repeats the check with the merged block's left neighbour. It stops when $\mu(B_1) \leq \mu(B_2) \leq \dots \leq \mu(B_r)$. $B(k)$ denotes the block that contains grid point k , and the solution assigns every point of a block the block's mean:

$$\bar{g}_l^{(i)}(a_k) = \mu(B(k)) = \frac{1}{\Omega_{B(k)}} \sum_{k' \in B(k)} \Omega_{k'} m_{l,k'}^{(i)}. \quad (13)$$

The partition depends on the current pooled values, so it is recomputed for every layer and every forward pass. We drop the indices i and l from B for readability. When the endpoints are covered (as on Newsroom), each forms a single-point block. Every shape satisfies $h(0) = 0$ and $h(1) = 1$, so $m_{l,1} = 0$ and $m_{l,K} = 1$ are the smallest and largest of the pooled values, and a strict violation involving them cannot occur. The partition is a function of the pooled values, $B = B(m_l^{(i)})$, but a discrete one. It is piecewise constant in $m_l^{(i)}$ and changes only where the means of two adjacent groups of points cross. Its derivative is therefore zero wherever it exists, and it needs no gradient. On each region where the partition is fixed, the projection is linear, $\bar{g}_l^{(i)} = A_B m_l^{(i)}$, where A_B is the block-averaging matrix. We therefore compute the partition from the current values without gradient, and differentiate the block means with the partition held fixed:

$$\frac{\partial \bar{g}_l^{(i)}(a_k)}{\partial m_{l,k'}^{(i)}} = \mathbb{I}[k' \in B(k)] \frac{\Omega_{k'}}{\Omega_{B(k)}}. \quad (14)$$

This is the exact Jacobian of weighted isotonic regression wherever the partition does not change under small perturbations of $m_l^{(i)}$. That holds everywhere except on a measure-zero set of pooled values, where two adjacent groups of points have exactly equal weighted means (cf. the unweighted case in Blondel et al., 2020). At such a point the projection is still continuous. Equation 9 has a unique solution, which is a projection onto the monotone cone under the Ω -weighted norm and hence 1-Lipschitz in $m_l^{(i)}$. Merging the two tied groups or not gives the same values, and only the one-sided Jacobians differ. Our strict-merging rule returns one of them, which is an element of the Clarke generalized Jacobian. Two cases follow. If

$m_l^{(i)}$ is already increasing, every block is a single point and the Jacobian is the identity. If a block has merged several points, the gradient arriving at any of them is shared among all members in proportion to their weights. This is the only place where the projection is non-local.

(3) *Pooling.* By the definition of $m_{l,k}^{(i)}$, $\partial m_{l,k}^{(i)} / \partial h_{i,l}(a_k) = \omega_{ik} / \Omega_k$. Combined with Equation 14, $\Omega_{k'}$ cancels:

$$\frac{\partial \bar{g}_l^{(i)}(a_k)}{\partial h_{i,l}(a_{k'})} = \mathbb{I}[k' \in B(k)] \frac{\omega_{ik'}}{\Omega_{B(k)}}. \quad (15)$$

Client i 's value at $a_{k'}$ therefore receives gradient only if $\omega_{ik'} > 0$, that is, only where the client has data. Its share is its evidence relative to the total evidence of the block. Where several clients overlap, each moves the pooled value in proportion to its weight.

(4) *Shape.* The gradient reaches $\psi_{i,l}$ through the closed-form derivative of the Kumaraswamy mixture $h_{i,l}(a_{k'})$, computed by automatic differentiation. In total,

$$\nabla_{\psi_{i,l}} \mathcal{L}_i = \sum_k G_{l,k} \sum_{k' \in B(k)} \frac{\omega_{ik'}}{\Omega_{B(k)}} \nabla_{\psi_{i,l}} h_{i,l}(a_{k'}), \quad (16)$$

where k runs over the covered grid points, and $G_{l,k}$ includes the gradient passed on by interpolated neighbours. The shape is parametric, so a step on $\psi_{i,l}$ also changes $h_{i,l}$ where the client has no data. Those values carry zero weight in $\bar{g}$, so they affect neither training nor inference. The projection is applied separately to each layer, so the loss at layer l trains only that layer's shapes.

Tie penalty. Where clients overlap, $\bar{g}$ depends only on the weighted average of their values. The individual curves are therefore not fully determined and could drift apart, creating dips that the projection must flatten. A small penalty, applied only where the client has data, ties each client's curve to the pooled one:

$$R_i = \frac{\lambda_{\max}}{LK} \sum_{l=1}^L \sum_{k=1}^K \omega_{ik} (h_{i,l}(a_k) - \text{sg}[\bar{g}_l^{(i)}(a_k)])^2, \quad (17)$$

where sg stops the gradient and L is the number of adapted matrices. The pooled curve is a constant in this term, so its gradient goes to the client's own values only: $\partial R_i / \partial h_{i,l}(a_k) = \frac{2\lambda_{\max}}{LK} \omega_{ik} (h_{i,l}(a_k) - \bar{g}_l^{(i)}(a_k))$. The local objective becomes $\mathcal{L}_i + R_i$, and the shape parameters

receive the sum of Equation 16 and this term. They are updated with AdamW together with the adapter and the direction.

Settings. We use $K = 21$ grid points, bandwidth $b = 0.2$, $\tau = 100$ and $\lambda_{\max} = 0.01$, with one shape per adapted matrix ($L = 112$ on Llama-3.2-1B) and no penalty toward the identity. The direction is averaged uniformly as before. Until round 5 the shapes are frozen at the identity, so $\bar{g}$ is the identity. The results in Section 2 use the shared calibration of Section 1.3.

1.5 Compared methods

All methods share the backbone, client data, α labels and adapter placement.

- **FED (ours).** The method above.
- **LOCAL.** The same model, loss and step budget, but nothing is aggregated. Each client learns its own direction and its own gain and warp from its own data.
- **B1: prompting.** On each client’s federated model with the direction removed ($B^D = 0$), the requested level is stated in the prompt as a 0–100 extractiveness target. The prompt has either no examples ($k = 0$) or $k = 3$ examples from the client’s own training data nearest to the target level. We also report $k = 3$ on the base model without an adapter.
- **B3: one-shot merge.** The uniform average of the LOCAL clients’ directions, attached to each client’s local adapter and calibration.
- **B4: federated activation steering.** Contrastive activation addition (Rimsky et al., 2024). Each client computes a mean-difference vector at the middle layer between its top- and bottom-quartile summaries, and the server averages these vectors. Each client then fits one gain on 30 development articles, and steering adds $s(2\alpha - 1) \|\bar{h}\| \hat{v}$ to the residual stream. Like B1, it runs on the client’s model with the direction removed.

B1 and B4 therefore keep the client’s private adapter and change only the control mechanism.

Held-out clients. A new client joins with its first n training pairs, for $n \in \{16, 64, 256, 1024, \text{all}\}$; the subsets are nested. Its α labels come from the participants’ reference F at no cost. We compare four ways for it to obtain control:

- **Frozen D .** The participants’ direction and shared calibration are frozen. The client trains a new private adapter with the participants’ objective, i.e. with steering active at each pair’s level.
- **Local D .** The same objective and number of steps, but the client also learns its own direction and calibration from zero.
- **Plug-in.** The adapter is trained by plain fine-tuning with steering off, and the frozen direction is attached afterwards.
- **Prompting.** The plug-in adapter without the direction, with the level in the prompt and three examples drawn from the n pairs.

All settings take $\text{clip}(\lceil 4n/8 \rceil, 200, 2000)$ steps: 200 for $n \leq 256$, 512 for $n = 1024$, and 2,000 for the full pool. Four epochs match the participants’ training budget.

1.6 Evaluation protocol and metrics

Protocol. During training, every 10th round’s checkpoint is evaluated on the development split (100 articles $\times$ 5 levels). We select the checkpoint with the lowest mean percentile error and report on test. At test time each client generates a summary for each of its test articles at $\alpha \in \{0, 0.25, 0.5, 0.75, 1\}$ with greedy decoding (at most 128 new tokens). Client-level metrics are averaged with equal client weights. We also report the worst client.

Control. The primary metric is the *percentile error* $|F(a(\hat{y}_\alpha)) - \alpha|$, averaged over articles and levels. It is bounded and scale-free. A model that always outputs the median scores 0.30 on this grid, so a useful control must stay below that. We split the error into in-support and out-of-support requests. *Reach* is the share of out-of-support outputs that leave the client’s own support in the requested direction. Ordering is measured by the per-article Spearman correlation between α and output density. Articles whose output does not change with α count as 0. The *near-tie* rate is the share of adjacent levels whose outputs are identical after normalizing quotes, whitespace and a trailing ellipsis, allowing a token-level edit ratio ≥ 0.95 . It guards against unchanged outputs that appear steerable, such as truncated leads cut at different positions.

Quality. Extractive summaries are easier to score as faithful, so quality is always compared with *real summaries at the same requested level* (binned by α). This follows the matched-extractiveness protocol of Ladhak et al. (2022). We report factual consistency with AlignScore-large (Zha et al., 2023), BERTScore F1 against the reference (Zhang et al., 2020), and length. On 25 articles per client and level, Qwen2.5-7B-Instruct also judges faithfulness at the sentence level, following FineSurE (Song et al., 2024), and rates relevance and coherence on a 1–5 scale; it is blind to method and level.

House style. To test whether a client’s outputs keep its publication’s style independently of extractiveness, we train a publication classifier on the participants’ real training summaries. It is logistic regression on character 2–5-gram and word 1–2-gram TF-IDF features. The training set is *density-matched*: within each of five α bins, every publication contributes the same number of summaries (at most 400), so extractiveness cannot identify the publication. *Attribution* is the share of a client’s in-support outputs that the classifier assigns to its own publication; chance is 0.125. We also report a *style-feature gap*, which is lower when outputs are closer to the client’s style. It is the absolute standardized mean difference between generated summaries and the client’s real summaries at a similar level (± 0.125 in α), averaged over 11 interpretable features. The features are sentence count, words per sentence, word length, quotes, ellipses, parentheses, numerals, capitalized words, first- and second-person pronouns, and question and exclamation marks.

Uncertainty. Method comparisons use the same test articles and levels. Per client, we compute a 95% paired bootstrap interval over articles (4,000 resamples) for the difference in percentile error, and count clients whose interval excludes zero.

2 Results

2.1 Collaboration improves control

Table 2 compares federated training with local training at equal data and steps. On Llama-3.2-1B, FED lowers the mean percentile error from 0.165 to 0.151 and the worst-client out-of-support error from 0.249 to 0.188. It also increases the per-article Spearman correlation from 0.906 to 0.933 and reduces near-ties from 0.246 to 0.204. FED is significantly better than LOCAL on four of eight

clients and significantly worse on none. On in-support requests alone, it is better on six. The result replicates on a second model family and scale. On Qwen3-8B, FED reduces the mean error from 0.177 to 0.156 and is significantly better on six of eight clients. The 2,000-pair setting shows the same direction: 0.162 for FED versus 0.180 for LOCAL, significantly better on six clients. That pair used per-client calibration with an offset.

The gains are largest where a client’s own data does not cover the request. Table 3 breaks the comparison down by client. The largest improvements occur for the two clients missing large parts of the scale, mainly on out-of-support requests. theguardian.com has almost no extractive summaries (support [0.04, 0.55]). Its out-of-support error falls by 0.101 on 1B (95% CI [−0.118, −0.084]) and by 0.137 on 8B. ny-post.com has almost no abstractive summaries (support [0.51, 0.95]). On 1B, its out-of-support error falls by 0.040 ([−0.051, −0.030]). In the 2,000-pair pair, theguardian.com’s mean output percentile at $\alpha = 0, \dots, 1$ is 0.18, 0.31, 0.49, 0.75, 0.90 under FED and 0.11, 0.37, 0.48, 0.57, 0.63 under LOCAL. Its local model stalls near the top of its own data, while the federated model reaches the 90th percentile of the shared scale. For clients whose data already spans nearly the whole scale, “out-of-support” refers only to the 5% tails, and FED is slightly worse there (up to +0.03 on 1B). Collaboration therefore helps most where a client’s data has gaps in coverage, consistent with the premise that clients with complementary coverage benefit from a shared direction.

Iterative federated training is needed, and the learned knob beats simpler control. Table 4 compares the method with alternative control mechanisms. Averaging independently learned local directions once (B3) gives an error of 0.184. That is worse than both FED and LOCAL, and FED is better on seven of eight clients. A merged direction does not match the adapters and calibrations it is attached to. Federated activation steering (B4) orders outputs only partially (Spearman 0.752). It lowers AlignScore out of support by 0.21 relative to real summaries at the same level, and lengthens summaries by 11.5 tokens. Stating the target level in the prompt (B1) does not steer density with this backbone: Spearman is near zero. Without examples, 70% of articles give near-identical summaries at $\alpha = 0$ and $\alpha = 1$. Fragment density is a cor-

Backbone	Method	Pct. err. ↓ (worst)	In ↓	Out ↓ (worst)	Reach ↑	Spearman ↑	Near-tie ↓	Better / worse
Llama-3.2-1B	FED	0.151 (0.199)	0.135	0.161 (0.188)	0.387	0.933	0.204	4 / 0
	LOCAL	0.165 (0.205)	0.148	0.165 (0.249)	0.363	0.906	0.246	0 / 4
Qwen3-8B	FED	0.156 (0.236)	0.130	0.171 (0.261)	0.364	0.917	0.201	6 / 1
	LOCAL	0.177 (0.231)	0.153	0.183 (0.281)	0.323	0.880	0.320	1 / 6

Table 2: Participants, test set (200 articles per client, 4,000 training pairs per client). Percentile error is averaged over clients, with the worst client in parentheses. In/Out: error on in-support and out-of-support requests. *Better / worse*: number of the 8 clients on which the method’s overall error is significantly lower / higher than the other row’s (95% paired bootstrap). A constant output scores 0.30.

Client	Support	Llama-3.2-1B				Qwen3-8B			
		FED	LOCAL	Δ overall	Δ out	FED	LOCAL	Δ overall	Δ out
theguardian.com	[0.04, 0.55]	0.133	0.200	−0.066*	−0.101*	0.127	0.227	−0.100*	−0.137*
nypost.com	[0.51, 0.95]	0.177	0.205	−0.028*	−0.040*	0.236	0.231	+0.005	+0.002
people.com	[0.03, 0.70]	0.152	0.160	−0.008	−0.005	0.146	0.169	−0.023*	−0.022*
aol.com	[0.06, 0.82]	0.132	0.138	−0.006	+0.020*	0.129	0.136	−0.008*	−0.026*
wsj.com	[0.04, 0.89]	0.144	0.151	−0.007*	+0.025*	0.124	0.145	−0.020*	−0.005
cbc.ca	[0.11, 0.95]	0.143	0.152	−0.009*	+0.008	0.129	0.151	−0.022*	+0.015*
reuters.com	[0.10, 0.97]	0.199	0.192	+0.007	+0.028*	0.221	0.202	+0.019*	+0.031*
forbes.com	[0.03, 0.99]	0.129	0.125	+0.003	+0.029*	0.133	0.157	−0.024*	+0.051*

Table 3: Per-client percentile error on test, with paired differences $\Delta = \text{FED} - \text{LOCAL}$ over all requests (overall) and over out-of-support requests (out); negative favours FED. *: 95% paired bootstrap interval excludes zero. Clients are ordered by the width of their support, narrowest first.

pus statistic that a model cannot easily estimate for its own output, so a learned control mechanism is needed.

Control comes without a quality cost. Table 5 shows that federated and local models have comparable consistency, BERTScore and judged faithfulness. Both stay close to real summaries requested at the same level. Absolute AlignScore varies with extractiveness even for real summaries: from 0.48 in the most abstractive bin to 0.84–0.87 in extractive bins. This is why we compare against the same-level reference. Out of support, the federated models are judged more relevant than the local ones (relevance gap +0.02 vs. −0.05 on 1B), and their outputs are 2–3 tokens longer than the reference. This reflects the coupling of density and length as they reach higher levels.

2.2 Private adaptation preserves house style

Collaboration should not erase what distinguishes a client’s writing. Table 6 measures publication style with a classifier trained so that extractiveness carries no information about the publication. The method’s outputs are attributed to their own publication 55.2% of the time, close to real test summaries (58.7%) and to locally trained models (55.3%). Sharing the direction therefore does not wash out private style. Making the adapter shared

as well, so that nothing is personalized, lowers attribution to 40.9% and nearly doubles the feature gap (0.190 vs. 0.108). Attribution drops for all eight clients, most for nypost.com (0.67 to 0.25) and theguardian.com (0.55 to 0.29). Two observations locate the style in the private adapter. Prompting on the client model, which keeps the adapter, retains attribution (0.556). The same prompts and in-context examples on the base model do not (0.274). Style is also orthogonal to the steered attribute: the method’s attribution stays between 0.52 and 0.56 across requested levels from $\alpha = 0$ to 1. The advantage over the shared adapter is present from round 10 onward, so it does not come from late-round memorization. The 8B models show the same pattern (0.559 federated, 0.577 local). A fully shared adapter extrapolates better out of support (0.136 vs. 0.161) at the cost of in-support control (0.177 vs. 0.135) and house style. Private adaptation is thus the choice that preserves client characteristics while retaining the collaboration gains above.

2.3 The learned control transfers to new clients

A direction that works only on the models it was co-trained with would be of limited use. Table 7 tests reuse on four clients that took no part in federation. With the participants’ direction and cali-

Method	Err. (worst)	In	Out	Reach	Spear.	End. NT	AlignScore in / out	Len. out	Ours better
FED (ours)	0.151 (0.199)	0.135	0.161	0.39	0.933	0.002	0.779 / 0.729	+2.3	–
B3 one-shot merge	0.184 (0.229)	0.149	0.220	0.26	0.878	0.009	0.788 / 0.708	−1.7	7/8
B4 federated CAA	0.263 (0.317)	0.212	0.312	0.22	0.752	0.000	0.594 / 0.476	+11.5	7/8
B1 prompt, $k = 0$	0.378 (0.412)	0.240	0.500	0.03	−0.002	0.704	0.648 / 0.650	−6.9	–
B1 prompt, $k = 3$	0.378 (0.401)	0.247	0.495	0.05	0.040	0.182	0.608 / 0.605	−7.2	8/8
B1 prompt, $k = 3$, base	0.341 (0.355)	0.243	0.467	0.12	0.077	0.000	0.609 / 0.631	+72	–

Table 4: Comparison with alternative control mechanisms (Llama-3.2-1B, test, 4,000 pairs per client). *End. NT*: share of articles whose outputs at $\alpha = 0$ and $\alpha = 1$ are near-identical. *Len. out*: tokens above real summaries at the same α , out-of-support. *Ours better*: clients on which FED has significantly lower error. *base*: base model without the private adapter.

Backbone	Method	AlignScore in / out	Gap in / out	BERTScore in / out	Len. gap in / out	Faithful in / out	Relevance gap out
Llama-3.2-1B	FED	0.779 / 0.729	−0.026 / +0.046	0.900 / 0.889	+2.0 / +2.3	0.82 / 0.79	+0.02
	LOCAL	0.790 / 0.694	−0.015 / +0.011	0.902 / 0.887	+2.7 / −0.5	0.84 / 0.79	−0.05
Qwen3-8B	FED	0.797 / 0.763	−0.007 / +0.080	0.903 / 0.893	+1.6 / +2.3	0.86 / 0.84	+0.09
	LOCAL	0.826 / 0.725	+0.022 / +0.042	0.904 / 0.891	+4.5 / −0.2	–	–

Table 5: Summary quality on test, in-support / out-of-support. *Gap*: AlignScore of generated minus real summaries requested at the same α . *Len. gap*: tokens above real summaries at the same α . Judge columns use 25 articles per client and level and are indicative. The local Qwen3-8B judge run had not finished at the time of writing.

Outputs	Attrib. $\uparrow$	$P(\text{own}) \uparrow$	Feat. gap $\downarrow$
Real test summaries	0.587	0.388	–
<i>Llama-3.2-1B</i>			
FED, private adapter (ours)	0.552	0.378	0.108
LOCAL	0.553	0.375	0.104
FED, shared adapter	0.409	0.295	0.190
B1 $k = 3$, client model	0.556	0.371	0.166
B1 $k = 3$, base model	0.274	–	0.666
<i>Qwen3-8B</i>			
FED, private adapter (ours)	0.559	–	0.106
LOCAL	0.577	–	0.115

Table 6: Extractiveness-controlled publication attribution on in-support test outputs (mean over 8 participants; chance 0.125). $P(\text{own})$: mean classifier probability of the client’s own publication. *Feat. gap*: style-feature gap to the client’s real summaries at the same level.

bration frozen, a new client that trains only its private adapter on 16 pairs reaches an error of 0.179. Learning its own direction from the same 16 pairs and steps gives 0.295, close to the constant-output level of 0.30. Frozen D is significantly better on all four held-out clients at every budget from 16 to 1,024 pairs (Table 8). At $n = 16$, the per-client differences range from -0.094 to -0.156 . With 16 pairs, the frozen direction already has a lower mean error than a locally learned direction with 1,024 pairs (0.192). At $n \leq 256$, both settings reach near-zero training loss in the same 200 steps, so local D ’s deficit reflects sample efficiency rather than under-training. The gap narrows as local data grows. At full data, frozen D is still better on two clients and ties on the other two, and its mean error (0.135) is below the participants’ own (0.151).

Joining requires training with the direction active. The frozen direction transfers through an adapter trained with it. Attaching the direction to an adapter fine-tuned without it (plug-in) fails at every budget (0.34–0.38). Its outputs at $\alpha = 0$ already sit high on the scale and are 26–37 tokens longer than real summaries at the same level. Joining therefore needs no new data type: the new client labels its pairs with the public reference F and trains its adapter with steering active, exactly as the participants did. Prompting stays above the constant-output reference.

Quality. Frozen and local D have equal summary quality. AlignScore gaps to the same-level reference are within 0.04 of each other at every n , BERTScore is 0.885–0.900 for both, and judged out-of-support relevance favours the frozen direction (-0.03 to $+0.03$ vs. -0.04 to -0.15). The exception is length: with 16 pairs, frozen D writes summaries 14 tokens longer than the reference out of support, which falls to 2–7 tokens from $n = 256$.

2.4 A shared calibration generalizes to under-covered clients

The global scale gives α the same meaning on every client, so the mapping from requested level to intervention strength can also be shared. Table 9 shows that sharing it gives the best mean error, worst client, out-of-support error and Spearman correlation. A per-client calibration can only be fitted where the client has data. For broadly covered clients, it brings small in-support gains (at most

n	Frozen D (in / out)	Local D (in / out)	Spearman fr. / loc.	Frozen better	Plug-in	Prompt	Align. gap out fr. / loc.
16	0.179 (0.203 / 0.153)	0.295 (0.289 / 0.299)	0.893 / 0.542	4/4	0.369	0.411	+0.05 / +0.06
64	0.170 (0.182 / 0.152)	0.260 (0.256 / 0.266)	0.922 / 0.735	4/4	0.376	0.388	+0.08 / +0.12
256	0.146 (0.148 / 0.143)	0.225 (0.233 / 0.213)	0.931 / 0.785	4/4	0.374	0.411	+0.06 / +0.06
1024	0.139 (0.141 / 0.136)	0.192 (0.209 / 0.165)	0.935 / 0.859	4/4	0.362	0.387	+0.07 / +0.04
all	0.135 (0.135 / 0.134)	0.151 (0.159 / 0.138)	0.932 / 0.925	2/4	0.343	–	+0.07 / +0.04

Table 7: Held-out clients (telegraph.co.uk, bbc.com, mashable.com, latimes.com), test, mean percentile error over the four clients as a function of the number n of local training pairs. *Frozen better*: clients on which frozen D has significantly lower error than local D (95% paired bootstrap); at full data the other two clients tie. *all* is 4,228–5,000 pairs.

Client	$n = 16$	$n = 256$	$n = 1024$	all
telegraph.co.uk	0.159 / 0.253	0.128 / 0.224	0.128 / 0.190	0.120 / 0.127
bbc.com	0.169 / 0.325	0.133 / 0.210	0.128 / 0.207	0.134 / 0.165
mashable.com	0.187 / 0.299	0.136 / 0.217	0.126 / 0.169	0.117 / 0.134
latimes.com	0.202 / 0.302	0.188 / 0.247	0.174 / 0.201	0.169 / 0.176

Table 8: Per held-out client percentile error, frozen D / local D .

Calib.	Off.	Err. (worst)	In	Out (worst)	Spear.	End. NT
Shared	no	0.151 (0.199)	0.135	0.161 (0.188)	0.933	0.002
Per-client	no	0.168 (0.320)	0.135	0.183 (0.365)	0.915	0.046
Shared	yes	0.158 (0.229)	0.141	0.167 (0.204)	0.920	0.002
Per-client	yes	0.155 (0.229)	0.134	0.165 (0.253)	0.926	0.009

Table 9: Shared versus per-client calibration g in federated training (Llama-3.2-1B, test). *Off.*: whether g includes an additive offset. *End. NT*: endpoint near-tie rate.

–0.015). It fails for the skewed client it is meant to help: nypost.com’s error rises by 0.143 and its out-of-support error from 0.188 to 0.365. Fitted only on $\alpha \in [0.51, 0.95]$, nypost.com’s private calibration learns a small gain and a strongly bent warp. Its outputs at $\alpha = 0, 0.25, 0.5, 0.75, 1$ sit at percentiles 0.64, 0.70, 0.76, 0.85, 0.90. With the shared calibration they span 0.18, 0.41, 0.71, 0.84, 0.90. The shared calibration also carries over to new clients. In the held-out setting, freezing it gives the same data-efficiency curve as training a private calibration on the new client’s pairs: 0.179, 0.170, 0.146, 0.139, 0.135 versus 0.183, 0.177, 0.151, 0.141, 0.132 for $n = 16, \dots, \text{all}$. A joining client therefore has no calibration parameters to fit.

Scope of the evidence. All results use one training seed and one held-out rotation, so the bootstrap intervals cover article sampling but not training or partition variance. The four held-out clients of rotation 0 have broad support. The held-out experiment therefore tests data efficiency more than transfer into coverage gaps. In the local baselines, the calibration gain ends at its upper bound of 4 for six

or seven of eight clients. The direction’s norm can compensate for this, but more slowly. The federated shared gain stays well inside the bound (1.93 on 1B, 2.31 on 8B). We keep the same bound for all runs.

References

- Mathieu Blondel, Olivier Teboul, Quentin Berthet, and Josip Djolonga. 2020. [Fast differentiable sorting and ranking](#). In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 950–959. PMLR.
- Max Grusky, Mor Naaman, and Yoav Artzi. 2018. [Newsroom: A dataset of 1.3 million summaries with diverse extractive strategies](#). In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*, pages 708–719. Association for Computational Linguistics.
- Faisal Ladhak, Esin Durmus, He He, Claire Cardie, and Kathleen McKeown. 2022. [Faithful or extractive? on mitigating the faithfulness-abstractiveness trade-off in abstractive summarization](#). In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1410–1421. Association for Computational Linguistics.
- Nina Rimskey, Nick Gabrieli, Julian Schulz, Meg Tong, Evan Hubinger, and Alexander Turner. 2024. [Steering Llama 2 via contrastive activation addition](#). In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 15504–15522. Association for Computational Linguistics.
- Hwanjun Song, Hang Su, Igor Shalyminov, Jason Cai, and Saab Mansour. 2024. [FineSurE: Fine-grained summarization evaluation using LLMs](#). In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 906–922. Association for Computational Linguistics.

- 689 Yuheng Zha, Yichi Yang, Ruichen Li, and Zhiting Hu.
690 2023. [AlignScore: Evaluating factual consistency](#)
691 [with a unified alignment function](#). In *Proceedings*
692 *of the 61st Annual Meeting of the Association for*
693 *Computational Linguistics (Volume 1: Long Papers)*,
694 pages 11328–11348. Association for Computational
695 Linguistics.
- 696 Tianyi Zhang, Varsha Kishore, Felix Wu, Kilian Q.
697 Weinberger, and Yoav Artzi. 2020. [BERTScore:](#)
698 [Evaluating text generation with BERT](#). In *Interna-*
699 *tional Conference on Learning Representations*.